

UNIVERSIDAD DE MANIZALES

FACULTAD DE CIENCIAS E INGENIERÍA

TRABAJO DE HABILITACIÓN

ARQUITECTURA DE SOFTWARE

Sistema de Gestión de Coworking con Reservas de Espacios y Membresías

Documento de Especificación Arquitectónica (SAD)

Presentado por:

Carlos Vargas

Docente:

Jose Ubaldo Carvajal

Arquitectura de Software

2026

Tabla de contenido

Capítulo 1. Especificación de Requisitos

- 1.1 Descripción del sistema
- 1.2 Módulos funcionales
- 1.3 Requisitos funcionales
- 1.4 Requisitos no funcionales
- 1.5 Restricciones
- 1.6 Partes interesadas
- 1.7 Objetivos arquitectónicos

Capítulo 2. Análisis y Riesgos

- 2.1 Casos de uso arquitectónicamente significativos
- 2.2 Esquema general del sistema
- 2.3 Riesgos arquitectónicos
- 2.4 Relación entre casos de uso y atributos de calidad

Capítulo 3. Decisión Arquitectónica

- 3.1 Arquitectura candidata A. Arquitectura monolítica
- 3.2 Arquitectura candidata B. Arquitectura basada en microservicios
- 3.3 Comparación de arquitecturas
- 3.4 ADR-001. Selección del estilo arquitectónico
- 3.5 Arquitectura seleccionada

Capítulo 4. Modelo de Vistas 4+1

- 4.1 Vista de escenarios
- 4.2 Vista lógica
- 4.3 Vista de procesos
- 4.4 Vista de desarrollo
- 4.5 Vista física

Capítulo 5. Especificación y Evaluación Completa

5.1 Vista de datos

5.2 Especificación de tamaño y rendimiento

5.3 Estrategia de despliegue

5.4 Evaluación arquitectónica mediante Mini-ATAM

Conclusión

Referencias

Capítulo 1. Especificación de Requisitos

1.1 Descripción del sistema

El Sistema de Gestión de Coworking corresponde a una aplicación web para administrar espacios de trabajo compartidos. El alcance funcional comprende el registro de usuarios, la administración de membresías, la reserva de puestos de trabajo, salas de reuniones y auditorios, y la validación del acceso a las instalaciones. También contempla la facturación, el intercambio de información con una pasarela de pagos y el envío de notificaciones sobre reservas, pagos y vencimientos. La consulta de disponibilidad y la centralización de estos procesos reducen las tareas manuales del personal administrativo.

1.2 Módulos funcionales

M01 Gestión de Usuarios

La gestión de usuarios reúne los datos de identificación y contacto necesarios para operar el coworking. Desde este módulo se consultan y actualizan los perfiles, y se asignan permisos de acuerdo con las funciones de cada usuario.

M02 Gestión de Membresías

Los planes, fechas de vigencia y estados de las membresías se administran en este módulo. También se registran las renovaciones y cancelaciones para que otras funciones, como reservas y control de acceso, trabajen con información actualizada.

M03 Gestión de Reservas

La gestión de reservas muestra los espacios disponibles para un intervalo determinado y permite reservar, modificar o cancelar puestos de trabajo, salas de reuniones y auditorios. Antes de confirmar una solicitud, debe comprobar que el espacio no haya sido asignado a otro usuario.

M04 Control de Acceso

En recepción, el control de acceso consulta si la persona tiene una membresía activa o una reserva vigente. Con esa verificación se autoriza o rechaza el ingreso y se conserva un registro de la decisión.

M05 Facturación

A partir de las membresías, reservas y servicios adicionales registrados, el componente de facturación calcula los valores aplicables y genera el documento correspondiente para cada usuario.

M06 Pagos

El procesamiento de pagos se encarga de remitir cada cobro a la pasarela externa y de asociar su respuesta con la factura y la membresía correspondientes. Si el proveedor no responde, la solicitud debe quedar identificada para evitar cobros duplicados.

M07 Notificaciones

Las confirmaciones de reserva, los recordatorios y las alertas de vencimiento se generan a partir de eventos del sistema. El resultado de cada envío se registra para que el personal pueda reconocer mensajes pendientes o fallidos.

1.3 Requisitos funcionales

Usuarios

RF-01 Registrar usuario.

RF-02 Modificar información de usuario.

RF-03 Consultar perfil.

Membresías

RF-04 Crear membresía.

RF-05 Renovar membresía.

RF-06 Cancelar membresía.

Reservas

RF-07 Crear reserva.

RF-08 Consultar disponibilidad.

RF-09 Cancelar reserva.

RF-10 Modificar reserva.

Acceso

RF-11 Validar acceso mediante membresía activa.

RF-12 Validar acceso mediante reserva vigente.

Facturación y pagos

RF-13 Generar factura.

RF-14 Procesar pago.

Notificaciones

RF-15 Enviar confirmación de reserva.

RF-16 Enviar recordatorio de reserva.

RF-17 Enviar alerta de vencimiento.

1.4 Requisitos no funcionales

RNF-01 Disponibilidad

El sistema deberá mantener una disponibilidad mínima de 99.9% mensual.

RNF-02 Rendimiento

El 95% de las consultas de disponibilidad deberán responder en menos de 2 segundos.

RNF-03 Escalabilidad

El sistema deberá soportar 500 usuarios concurrentes.

RNF-04 Seguridad

Toda comunicación deberá realizarse mediante HTTPS TLS 1.2 o superior.

RNF-05 Recuperación

El sistema deberá recuperar la operación en menos de 30 minutos ante una falla crítica.

RNF-06 Mantenibilidad

Nuevos módulos podrán agregarse sin modificar componentes existentes.

RNF-07 Auditoría

El sistema deberá registrar los eventos relacionados con accesos, reservas, pagos y modificaciones de información, de modo que sea posible consultar quién realizó cada acción y cuándo ocurrió.

RNF-08 Respaldo de información

El sistema deberá realizar copias de seguridad periódicas de la información operativa para permitir la recuperación ante incidentes o pérdida de datos.

RNF-09 Continuidad operativa

El sistema deberá contar con mecanismos de recuperación que permitan restablecer la operación ante fallas críticas dentro del tiempo definido en el requisito RNF-05.

1.5 Restricciones

Restricción 1

El sistema deberá desplegarse sobre servicios de computación en la nube.

Restricción 2

El sistema deberá integrarse con una pasarela de pagos externa.

Restricción 3

Los usuarios deberán acceder desde navegador web.

Restricción 4

Los datos financieros deberán almacenarse cifrados y su consulta deberá limitarse a usuarios autorizados.

Restricción 5

El sistema deberá exponer o consumir interfaces de programación de aplicaciones (API) para intercambiar información con servicios externos.

Restricción 6

La información de usuarios, pagos y membresías deberá almacenarse de acuerdo con políticas de protección y seguridad de datos.

1.6 Partes interesadas

Parte interesada	Interés
Usuario del coworking	Reservar espacios
Administrador	Gestionar operación
Personal de recepción	Validar accesos
Área financiera	Facturación
Proveedor de pagos	Procesar transacciones

Equipo de soporte y operación	Mantener disponible el servicio y restablecerlo ante fallas
-------------------------------	---

1.7 Objetivos arquitectónicos

OA-01. Mantener disponible el sistema para registrar y consultar reservas y membresías.

OA-02. Permitir la incorporación de nuevos módulos sin modificar los componentes existentes.

OA-03. Proteger la información de usuarios y transacciones mediante mecanismos de seguridad y control de acceso.

OA-04. Permitir la integración con servicios externos de pago y notificación.

OA-05. Restablecer la operación en menos de 30 minutos ante una falla crítica mediante copias de seguridad y procedimientos de recuperación.

Capítulo 2. Análisis y Riesgos

2.1 Casos de uso arquitectónicamente significativos

Caso de Uso 1. Registro de Usuario

Descripción

El usuario crea una cuenta mediante el registro de los datos obligatorios. El sistema valida la información recibida antes de almacenarla.

Justificación arquitectónica

El flujo requiere validación de entradas, protección de datos personales y asignación controlada de credenciales y permisos. Un error en estas operaciones puede producir cuentas duplicadas o accesos no autorizados.

Caso de Uso 2. Adquisición o Renovación de Membresía

Descripción

El usuario selecciona un plan y solicita su adquisición o renovación. El sistema procesa el pago y actualiza el estado y la vigencia de la membresía según el resultado de la transacción.

Justificación arquitectónica

El flujo depende de la pasarela de pagos y exige mantener correspondencia entre la transacción, la factura y el estado de la membresía. El sistema debe garantizar que cada pago se procese una sola vez y que la vigencia de la membresía corresponda al resultado de la transacción.

Caso de Uso 3. Reserva de Espacio de Trabajo

Descripción

El usuario consulta la disponibilidad de un puesto de trabajo, una sala de reuniones o un auditorio y registra la reserva para el intervalo seleccionado.

Justificación arquitectónica

La operación requiere consultar disponibilidad y confirmar la reserva como una unidad consistente. Las solicitudes concurrentes sobre el mismo espacio no deben producir reservas superpuestas.

Caso de Uso 4. Control de Acceso a las Instalaciones

Descripción

El personal de recepción solicita la validación del usuario. El sistema autoriza o rechaza el ingreso según la vigencia de la membresía o de la reserva presentada.

Justificación arquitectónica

La validación debe permanecer disponible durante el horario de operación y consultar información vigente. Una respuesta incorrecta puede autorizar un ingreso no permitido o impedir el acceso de un usuario habilitado.

Caso de Uso 5. Generación de Facturación y Notificaciones

Descripción

El sistema genera la factura correspondiente a los servicios registrados y solicita el envío de confirmaciones, recordatorios o alertas según el evento producido.

Justificación arquitectónica

El flujo coordina información de reservas, membresías, pagos y facturación. El envío de mensajes depende de un servicio externo; por esta razón, el sistema debe registrar cada solicitud y la respuesta recibida para identificar entregas fallidas o pendientes.

2.2 Esquema general del sistema

Descripción general

El sistema se divide en módulos de usuarios, membresías, reservas, control de acceso, facturación, pagos y notificaciones. La interfaz web recibe las solicitudes de los usuarios y las dirige al módulo correspondiente. La información operativa se conserva en la base de datos. Las operaciones de cobro y envío de mensajes se realizan mediante servicios externos, por lo que su disponibilidad y sus respuestas deben tratarse de forma independiente del procesamiento interno.

Componentes principales

- Gestión de Usuarios
- Gestión de Membresías
- Gestión de Reservas
- Control de Acceso
- Facturación
- Pagos
- Notificaciones
- Base de Datos
- Pasarela de Pagos Externa

2.3 Riesgos arquitectónicos

Riesgo	Impacto	Estrategia de Mitigación
Indisponibilidad de la pasarela de pagos	Imposibilidad de procesar transacciones	Implementar reintentos y almacenamiento temporal de solicitudes
Sobrecarga durante horarios de alta demanda	Degradación del rendimiento	Escalamiento horizontal de servicios críticos
Acceso no autorizado a información sensible	Pérdida de confidencialidad	Autenticación, autorización y cifrado de comunicaciones
Pérdida de información por fallas de infraestructura	Afectación de la operación	Copias de seguridad periódicas y procedimientos de recuperación

Duplicidad de reservas	Inconsistencias operativas	Validación de disponibilidad en tiempo real
Dependencia de servicios externos	Interrupción de funcionalidades	Manejo de errores y mecanismos de recuperación
Recuperación tardía después de una falla crítica	Interrupción prolongada del servicio	Procedimientos documentados de recuperación y restauración desde copias de seguridad

2.4 Relación entre casos de uso y atributos de calidad

Caso de uso	Atributos de calidad involucrados
Registro de usuario	Seguridad, disponibilidad
Adquisición de membresía	Seguridad, interoperabilidad
Reserva de espacio	Rendimiento, escalabilidad
Control de acceso	Seguridad, disponibilidad
Facturación y notificaciones	Interoperabilidad, mantenibilidad

Capítulo 3. Decisión Arquitectónica

3.1 Arquitectura candidata A. Arquitectura monolítica

Descripción

La arquitectura monolítica concentra todos los módulos funcionales del sistema dentro de una única aplicación desplegable. Los módulos de usuarios, membresías, reservas, control de acceso, facturación, pagos y notificaciones comparten la misma base de código y la misma unidad de despliegue.

Ventajas

- El equipo puede construir y probar la primera versión sin coordinar despliegues entre varios servicios.

- La aplicación se publica como una sola unidad, lo que reduce la configuración inicial de infraestructura.
- Las operaciones que abarcan reservas, pagos y membresías pueden manejarse con transacciones internas más sencillas.
- Durante las primeras etapas, el seguimiento de errores se concentra en una única aplicación.

Desventajas

- Un aumento de consultas de disponibilidad obligaría a escalar también los módulos que no presentan la misma demanda.
- Los cambios en pagos o facturación pueden exigir validar y desplegar nuevamente toda la aplicación.
- Una falla no contenida podría afectar al mismo tiempo las reservas y la validación de ingreso en recepción.
- A medida que se incorporen nuevos servicios del coworking, la base de código será más difícil de mantener y probar.

3.2 Arquitectura candidata B. Arquitectura basada en microservicios

Descripción

La arquitectura basada en microservicios divide la solución en servicios independientes responsables de funciones específicas del negocio. Cada servicio dispone de responsabilidades claramente delimitadas y puede evolucionar de manera independiente.

Los servicios considerados para este sistema son:

- Servicio de Usuarios
- Servicio de Membresías
- Servicio de Reservas
- Servicio de Control de Acceso
- Servicio de Facturación
- Servicio de Pagos
- Servicio de Notificaciones

Ventajas

- El servicio de reservas puede aumentar su capacidad en horas de alta demanda sin escalar el resto de la solución.

- Los cambios en la integración con la pasarela pueden desplegarse sin intervenir directamente el control de acceso.
- Una falla del servicio de notificaciones no tiene que impedir la creación de una reserva o la validación de ingreso.
- Cada servicio puede evolucionar según las reglas de su proceso, siempre que conserve los contratos de integración.
- La separación facilita incorporar nuevos tipos de espacios o proveedores externos sin concentrar todos los cambios en una sola aplicación.

Desventajas

- La operación exige administrar varios despliegues, configuraciones y versiones de servicio.
- Para localizar una falla es necesario correlacionar registros y métricas distribuidos entre distintos componentes.
- Procesos como adquirir una membresía requieren coordinar pagos, facturación y actualización de vigencia sin depender de una única transacción local.
- La infraestructura y la automatización inicial demandan más tiempo y experiencia técnica que una aplicación monolítica.

3.3 Comparación de arquitecturas

Criterio	Monolítica	Microservicios
Complejidad de desarrollo inicial	Baja complejidad inicial	Mayor complejidad inicial
Escalabilidad	Escalamiento completo de la aplicación	Escalamiento independiente
Mantenibilidad	Limitada a medida que crece el sistema	Favorece la evolución modular
Disponibilidad	Una falla puede afectar toda la aplicación	Aislamiento de fallas entre servicios
Integración con sistemas externos	Menor flexibilidad	Mayor flexibilidad
Crecimiento futuro	Limitado	Adecuado para crecimiento progresivo

Resultado de la evaluación

La alternativa monolítica sería suficiente para una primera versión con alcance y equipo reducidos, porque simplifica el desarrollo y la puesta en producción. Sin embargo, el sistema debe mantener disponible el control de acceso, atender picos de consultas de reservas e integrar proveedores externos que pueden fallar de forma independiente. Por estas razones, y asumiendo que el equipo dispone de prácticas de despliegue y monitoreo distribuido, la arquitectura de microservicios ofrece un ajuste más sólido a los objetivos planteados.

3.4 ADR-001. Selección del estilo arquitectónico

Selección de una arquitectura basada en microservicios para el Sistema de Gestión de Coworking.

Contexto

El sistema debe gestionar reservas, membresías, pagos, facturación y control de acceso. También debe soportar integración con servicios externos, crecimiento funcional y mecanismos de continuidad operativa.

Decisión

Adoptar una arquitectura basada en microservicios para implementar los módulos funcionales principales del sistema.

Justificación

La decisión se fundamenta en tres necesidades concretas. Primero, las consultas y confirmaciones de reservas pueden crecer en horarios específicos y deben escalar sin aumentar innecesariamente los recursos de facturación. Segundo, una interrupción de la pasarela de pagos o del proveedor de notificaciones no debería bloquear el control de acceso ni la consulta de espacios. Tercero, los módulos evolucionan con reglas distintas y requieren despliegues que reduzcan el impacto sobre funciones ya operativas. Esta separación introduce más trabajo de integración y monitoreo, por lo que su adopción supone contar con automatización de despliegues, trazabilidad entre servicios y procedimientos de recuperación.

Consecuencias Positivas

- Mayor capacidad de escalamiento.
- Menor acoplamiento entre módulos.
- Facilidad para incorporar nuevas funcionalidades.

- Mayor aislamiento de fallas.

Consecuencias Negativas

- Mayor complejidad de administración.
- Necesidad de monitoreo distribuido.
- Incremento en las comunicaciones entre servicios.

3.5 Arquitectura seleccionada

Se selecciona una arquitectura basada en microservicios porque permite separar los procesos que presentan ritmos de cambio, cargas y dependencias diferentes dentro del coworking.

Usuarios, membresías, reservas, control de acceso, facturación, pagos y notificaciones se implementan como servicios con responsabilidades delimitadas. En la operación diaria, esta separación permite asignar más capacidad al proceso de reservas, mantener la validación de ingreso cuando falle un proveedor externo y desplegar cambios de pagos sin sustituir toda la solución.

Los servicios intercambian información mediante API y, en procesos que no requieren respuesta inmediata, mediante mensajes. Cada servicio administra los datos propios de su responsabilidad y comparte únicamente la información necesaria a través de contratos definidos; así se evita que varios módulos modifiquen directamente las mismas tablas. Las copias de seguridad, el cifrado y los controles de acceso se aplican de acuerdo con la criticidad de los datos, mientras que los registros distribuidos permiten seguir una operación desde la reserva hasta el pago o la notificación.

Capítulo 4. Modelo de Vistas 4+1

El modelo 4+1 permite describir la arquitectura desde perspectivas complementarias. En este sistema, las vistas se utilizan para relacionar las acciones de usuarios y personal del coworking con los servicios que las ejecutan, el orden en que se comunican, la forma en que se organiza el código y la infraestructura donde se despliega la solución. De esta manera, cada decisión puede rastrearse hasta los requisitos y riesgos identificados en los capítulos anteriores.

4.1 Vista de escenarios

Esta vista parte de las situaciones que tienen mayor efecto sobre la arquitectura. No reúne todas las funciones del sistema, sino los recorridos que obligan a coordinar varios servicios o que deben permanecer disponibles durante la operación diaria del coworking.

Actores y escenarios principales

- **Usuario del coworking:** registra su cuenta, consulta espacios disponibles, adquiere o renueva una membresía y crea, modifica o cancela una reserva.
- **Administrador:** consulta usuarios, planes, reservas, facturas y resultados de pagos para mantener la operación actualizada.
- **Personal de recepción:** valida una membresía activa o una reserva vigente antes de autorizar el ingreso.
- **Pasarela de pagos:** recibe la solicitud de cobro y devuelve el resultado de la transacción.
- **Proveedor de notificaciones:** entrega confirmaciones, recordatorios y alertas generadas por el sistema.

El escenario de reserva comienza cuando el usuario consulta la disponibilidad para una fecha y un intervalo. El sistema verifica que el espacio continúe libre antes de confirmar la operación y registra la reserva sin permitir cruces con otra solicitud. Si el tipo de espacio o servicio exige un cobro adicional, el pago se procesa antes de dejar la reserva en estado confirmado; en los demás casos, basta con validar la membresía vigente. Después de la confirmación se genera la notificación correspondiente.

El escenario de acceso utiliza información ya registrada. Recepción consulta la vigencia de la membresía o de la reserva y obtiene una respuesta de autorización o rechazo. Este flujo no depende de que la pasarela de pagos o el proveedor de notificaciones estén disponibles en ese momento, lo que evita que una falla externa detenga el ingreso de usuarios habilitados.

Relación arquitectónica

Los escenarios muestran por qué se separan reservas, membresías, pagos, facturación, control de acceso y notificaciones. También sirven como referencia para comprobar las demás vistas: la vista lógica identifica los servicios responsables, la vista de procesos detalla su comunicación y la vista física define dónde se ejecutan.

Figura 1. Diagrama de casos de uso del Sistema de Gestión de Coworking.

Ver Figura 1 en el repositorio GitHub.

En la Figura 1 se concentran los actores y recorridos que condicionan el diseño. Además de las acciones del usuario, aparecen las tareas del personal administrativo y de recepción, junto con la comunicación con los proveedores externos de pagos y notificaciones. Estos escenarios permiten delimitar los servicios de reservas, membresías,

facturación y control de acceso, y relacionarlos con los riesgos de indisponibilidad externa y duplicidad de reservas identificados en la sección 2.3.

4.2 Vista lógica

La vista lógica divide el sistema de acuerdo con las responsabilidades del negocio. Cada servicio conserva las reglas de su dominio y expone operaciones mediante API, evitando que otros componentes modifiquen directamente sus datos.

Responsabilidades de los servicios

- **Usuarios:** administra perfiles, credenciales y datos de contacto.
- **Membresías:** controla planes, vigencias, renovaciones, cancelaciones y estado de cada afiliación.
- **Reservas:** consulta disponibilidad y evita que dos usuarios confirmen el mismo espacio en un intervalo coincidente.
- **Control de acceso:** consulta membresías y reservas vigentes para responder a recepción sin incorporar reglas de facturación.
- **Facturación:** calcula valores y genera los documentos asociados con membresías, reservas y servicios adicionales.
- **Pagos:** encapsula la comunicación con la pasarela, registra el resultado de cada intento y evita que una solicitud repetida produzca un cobro duplicado.
- **Notificaciones:** recibe eventos de negocio y gestiona confirmaciones, recordatorios y alertas sin bloquear la operación que los originó.

Las comunicaciones que necesitan respuesta inmediata, como consultar disponibilidad o validar un acceso, se realizan mediante API. Las tareas que pueden ejecutarse después de la operación principal, como enviar una confirmación, se publican a través del servicio de mensajería. Esta combinación reduce la dependencia directa entre reservas y notificaciones y permite reintentar un envío sin repetir la reserva.

La separación también facilita aplicar controles distintos según el tipo de información. Los datos personales y financieros quedan restringidos a los servicios autorizados, mientras que cada operación relevante genera registros de auditoría para reconstruir quién realizó la acción y cuándo ocurrió.

Figura 2. Diagrama de componentes y relaciones entre servicios.

Ver Figura 2 en el repositorio GitHub.

El diagrama de la Figura 2 distribuye las responsabilidades entre los servicios de usuarios, membresías, reservas, control de acceso, pagos, facturación y notificaciones. Las operaciones que requieren respuesta inmediata se resuelven mediante API; los eventos

quedan para tareas posteriores, como el envío de confirmaciones, sin mantener bloqueado el proceso original. Esta decisión aporta al desacoplamiento previsto en el RNF-06 y reduce el efecto de las dependencias externas señalado en la sección 2.3.

4.3 Vista de procesos

La vista de procesos explica el orden de las llamadas y los cambios de estado durante una operación crítica. Para este sistema se toma como referencia la reserva de un espacio, porque combina validaciones de membresía, control de concurrencia, un posible pago y el envío posterior de una notificación.

Secuencia de reserva

1. El usuario consulta los espacios disponibles para una fecha y un intervalo.
2. El servicio de Reservas obtiene la disponibilidad y verifica que no exista una reserva superpuesta.
3. El servicio de Membresías confirma si el usuario tiene un plan activo y si este cubre el espacio solicitado.
4. Cuando la membresía cubre la reserva, el servicio de Reservas la confirma sin solicitar un pago adicional. Si existe un valor por cobrar, la reserva queda temporalmente pendiente mientras el servicio de Pagos procesa la transacción.
5. Con un pago aprobado, el servicio de Reservas cambia el estado a confirmado y Facturación registra el concepto correspondiente. Si el pago falla o vence el tiempo de espera, la reserva no se confirma y el espacio vuelve a estar disponible.
6. Finalmente se publica un evento para que Notificaciones envíe la confirmación. Si el proveedor de mensajería no responde, el envío puede reintentarse sin repetir el pago ni crear otra reserva.

Para evitar duplicidades, la comprobación final de disponibilidad y la creación de la reserva deben ejecutarse como una operación consistente. Las solicitudes de pago también deben incluir un identificador único, de modo que un reintento técnico no produzca dos cobros. Los tiempos de respuesta, errores y cambios de estado se registran con un identificador de correlación común para seguir la operación entre servicios.

Este flujo concreta los atributos de rendimiento, disponibilidad y escalabilidad: Reservas puede aumentar su capacidad en horas de alta demanda, mientras que Pagos y Notificaciones pueden manejar fallas externas sin detener la consulta de espacios ni el control de acceso.

Figura 3. Diagrama de secuencia del proceso de reserva de espacio.

Ver Figura 3 en el repositorio GitHub.

El orden de interacción representado en la Figura 3 inicia con la consulta de disponibilidad y la validación de la membresía. Cuando existe un valor adicional, Pagos remite la transacción a la pasarela y Facturación registra el concepto correspondiente. Solo después de confirmar la reserva se publica el evento consumido por Notificaciones. Con esta secuencia, una falla en el envío del mensaje no obliga a repetir la reserva ni el cobro, y se mitiga el riesgo de transacciones duplicadas descrito en la sección 2.3.

4.4 Vista de desarrollo

La vista de desarrollo traduce los límites funcionales de la vista lógica en unidades que el equipo puede construir, probar y desplegar de manera independiente. Se propone un repositorio por servicio para que los cambios en pagos, reservas o notificaciones no obliguen a publicar nuevamente toda la solución.

Organización de repositorios

- **coworking-users-service:** perfiles, autenticación y permisos.
- **coworking-memberships-service:** planes, vigencias, renovaciones y cancelaciones.
- **coworking-reservations-service:** disponibilidad, creación, modificación y cancelación de reservas.
- **coworking-access-service:** reglas de autorización de ingreso y registro de accesos.
- **coworking-billing-service:** cálculo de conceptos y generación de facturas.
- **coworking-payments-service:** integración con la pasarela y seguimiento de transacciones.
- **coworking-notifications-service:** plantillas, colas de envío y estado de entrega.

Cada repositorio debe incluir el código del servicio, pruebas automatizadas, configuración de despliegue y documentación de su API o de los eventos que publica y consume. Los contratos compartidos deben versionarse para evitar que una actualización de un servicio rompa a los demás. La información sensible, como cadenas de conexión o credenciales de proveedores, no se almacena en el código fuente y se obtiene desde un mecanismo seguro de configuración.

El trabajo se integra mediante ramas y solicitudes de cambio revisadas antes de llegar a la rama principal. A partir de esta rama, una canalización automatizada de integración continua compila, ejecuta pruebas y genera el artefacto que se despliega en cada ambiente. Esta organización favorece la mantenibilidad, pero conserva la trazabilidad entre una modificación, su versión y el servicio publicado.

Figura 4. Mapa de repositorios, contratos e integración continua.

Ver Figura 4 en el repositorio GitHub.

Desde la perspectiva de desarrollo, la Figura 4 organiza cada servicio en un repositorio que contiene código, pruebas y configuración de despliegue. Los contratos compartidos definen la información intercambiada entre componentes, mientras que la canalización valida los cambios antes de generar una versión. Esta organización responde al RNF-06, ya que un servicio puede modificarse y publicarse sin reconstruir toda la solución.

4.5 Vista física

La vista física ubica los componentes en Azure y muestra los límites entre el acceso público, el procesamiento de negocio, la persistencia y las integraciones externas. La distribución propuesta busca que una falla en notificaciones o pagos no afecte las funciones básicas de reservas y control de acceso.

Infraestructura propuesta

- **Azure Application Gateway:** recibe las conexiones HTTPS, aplica las reglas de publicación y dirige las solicitudes hacia la aplicación.
- **Azure App Service:** cada servicio puede desplegarse de forma independiente para reducir el impacto de las actualizaciones. Los servicios con mayor demanda, especialmente Reservas, pueden ampliar su capacidad sin escalar todos los componentes al mismo tiempo.
- **Azure SQL Database:** Azure SQL Database constituye un componente crítico para el rendimiento de la solución. El aumento de usuarios concurrentes puede requerir la ampliación de recursos de base de datos o ajustes sobre las consultas con mayor volumen de ejecución para mantener los tiempos de respuesta definidos.
- **Azure Service Bus:** transporta eventos de reservas, pagos y vencimientos hacia Notificaciones y otros procesos que no requieren una respuesta inmediata.
- **Azure Key Vault:** protege secretos, claves y credenciales utilizadas para conectarse con la base de datos y los proveedores externos.
- **Azure Monitor y Application Insights:** centralizan métricas, errores y trazas para detectar fallas y seguir una operación entre varios servicios.
- **Servicios externos:** corresponden a la pasarela de pagos y al proveedor de notificaciones; ambos se consumen a través de conexiones cifradas y con control de tiempos de espera y reintentos.

Application Gateway constituye el punto de entrada y evita exponer directamente los servicios internos. La aplicación se comunica con la base de datos, Service Bus y Key Vault mediante conexiones controladas. Los registros de auditoría y telemetría permiten

comprobar accesos, reservas y pagos, además de identificar el componente responsable cuando una transacción falla.

Para cumplir el objetivo de recuperación menor a 30 minutos, la base de datos debe contar con copias de seguridad verificadas y el despliegue de los servicios debe poder repetirse a partir de su configuración automatizada. La restauración no se limita a recuperar datos: también debe contemplar la publicación de los servicios, la validación de sus conexiones y una prueba de los flujos de reserva y acceso antes de reabrir la operación.

La distribución física responde de forma directa a los requisitos de disponibilidad, seguridad, escalabilidad y recuperación definidos en los RNF-01, RNF-03, RNF-04 y RNF-05. También mantiene la trazabilidad con las vistas anteriores: cada servicio desplegado corresponde a un componente lógico, ejecuta una parte de los procesos descritos y se construye desde su repositorio.

Figura 5. Diagrama de despliegue de la solución en Azure.

Ver Figura 5 en el repositorio GitHub.

En el despliegue de la Figura 5, Application Gateway recibe las solicitudes externas y las dirige a los servicios publicados en App Service. Azure SQL Database conserva la información operativa y Service Bus transporta los eventos que pueden procesarse posteriormente. Key Vault protege las credenciales, mientras que Azure Monitor y Application Insights reúnen métricas y trazas. La pasarela de pagos y el proveedor de notificaciones permanecen como dependencias externas, por lo que se mantienen los controles de tiempo de espera y reintento previstos en la sección 2.3.

Capítulo 5. Especificación y Evaluación Completa

5.1 Vista de datos

La vista de datos identifica la información que sostiene la operación del coworking y establece qué servicio puede administrarla. Las entidades se distribuyen de acuerdo con las responsabilidades definidas para cada componente. Por ejemplo, Reservas controla la disponibilidad y asignación de espacios, Pagos conserva el resultado de las transacciones realizadas mediante la pasarela externa y Control de Acceso registra cada validación de ingreso.

Entidades principales

- Usuario
- Membresía

- Reserva
- Espacio
- Factura
- Pago
- Notificación
- Registro de Acceso

Relaciones principales

- Un usuario puede tener varias membresías a lo largo del tiempo.
- Un usuario puede registrar múltiples reservas.
- Una reserva corresponde a un único espacio.
- Una factura puede contener uno o varios conceptos asociados a membresías o reservas.
- Un pago se relaciona con una factura.
- Una notificación puede originarse por una reserva, un pago o una membresía.
- Un registro de acceso corresponde a una validación realizada para un usuario determinado.

Consideraciones arquitectónicas

Los datos operativos se almacenan en una base de datos relacional para conservar la integridad de usuarios, membresías, reservas, facturas y pagos. Aunque la solución utiliza una plataforma de datos común, cada servicio modifica únicamente la información de su dominio y comparte los datos necesarios mediante API o eventos definidos. Esta regla evita que varios componentes actualicen directamente las mismas tablas y reduce el riesgo de inconsistencias.

Relación con atributos de calidad

La separación de los datos por dominio limita el acceso innecesario a información personal o financiera y permite reconocer qué servicio efectuó cada modificación. Los registros de auditoría conservan el usuario, la fecha y el resultado de las operaciones relevantes. A su vez, las copias de seguridad y los procedimientos de restauración permiten recuperar la información ante una falla sin perder la trazabilidad de reservas, pagos o accesos.

Figura 6. Modelo conceptual y relaciones de datos del sistema.

Ver Figura 6 en el repositorio GitHub.

Las entidades y relaciones de la Figura 6 permiten reconstruir el recorrido de la información dentro del coworking. Un usuario puede tener membresías, crear reservas y registrar accesos; las reservas se asignan a espacios específicos y pueden originar conceptos de facturación y notificaciones; los pagos quedan asociados con la factura correspondiente. Esta trazabilidad respalda los requisitos de auditoría y respaldo definidos en los RNF-07 y RNF-08.

5.2 Especificación de tamaño y rendimiento

Para evaluar la arquitectura se adopta un escenario inicial de operación media, con crecimiento progresivo de usuarios y espacios administrados. Los valores siguientes no corresponden a mediciones de una instalación existente; funcionan como referencia para estimar la carga y comprobar si las decisiones propuestas pueden cumplir los requisitos de rendimiento.

Supuestos operativos

El escenario considera 5.000 usuarios registrados, 1.000 membresías activas y un promedio de 300 reservas diarias. En periodos de alta demanda se estima una concurrencia máxima de 500 usuarios y hasta 50 validaciones de acceso simultáneas durante los horarios de ingreso.

Objetivos de rendimiento

- Consultas de disponibilidad en menos de 2 segundos para el 95 % de las solicitudes.
- Validación de acceso en menos de 1 segundo.
- Confirmación de reservas en menos de 3 segundos cuando no exista procesamiento de pago.
- Registro de pagos según los tiempos de respuesta suministrados por la pasarela externa.

Decisiones asociadas

Los servicios de Reservas y Control de Acceso pueden aumentar su capacidad de forma independiente cuando la demanda lo requiera. No obstante, esta medida no garantiza por sí sola el cumplimiento de los tiempos de respuesta, porque las consultas de disponibilidad, las validaciones de membresías y la confirmación de reservas también

dependen de Azure SQL Database. Antes de pasar a producción deben ejecutarse pruebas de carga para medir ambos componentes, revisar las consultas de mayor frecuencia y ajustar la capacidad disponible sin comprometer la consistencia de las reservas.

Relación con atributos de calidad

- Rendimiento.
- Escalabilidad.
- Disponibilidad.

5.3 Estrategia de despliegue

Para publicar nuevas versiones se propone una estrategia Blue-Green. Esta alternativa permite probar los cambios en un ambiente paralelo antes de dirigir hacia él las solicitudes de los usuarios.

Justificación

Reservas, pagos y control de acceso forman parte de la operación diaria. Una versión defectuosa podría impedir la asignación de espacios, dejar transacciones en estados inconsistentes o bloquear el ingreso de usuarios habilitados. Mantener temporalmente dos ambientes reduce este riesgo porque la nueva versión puede validarse antes del cambio de tráfico y la versión anterior continúa disponible como alternativa de retorno.

Funcionamiento

El ambiente Blue mantiene la versión que atiende a los usuarios, mientras que el ambiente Green recibe la nueva versión. En Green se comprueban la creación y cancelación de reservas, la consulta de membresías, la validación de accesos y la comunicación con la pasarela. Solo después de superar estas pruebas se redirige el tráfico. Si aparece una falla, las solicitudes regresan a Blue sin reconstruir la versión anterior.

Beneficios y limitaciones

La estrategia reduce el tiempo de interrupción y permite volver rápidamente a la versión estable si las pruebas posteriores al cambio detectan un problema. Su costo es mantener ambos ambientes activos durante la actualización, con el aumento temporal de recursos que esto implica. Además, las modificaciones de base de datos deben ser compatibles con las dos versiones mientras termina la transición; de lo contrario, el retorno a Blue podría recuperar la aplicación, pero no el estado anterior de los datos.

Relación con atributos de calidad

La disponibilidad y la continuidad operativa mejoran porque la versión anterior permanece disponible durante la liberación. La recuperación depende de que el cambio de tráfico esté automatizado, de que existan criterios claros para cancelar el despliegue y de que las migraciones de datos hayan sido diseñadas para permitir el retorno.

Figura 7. Estrategia Blue-Green para la publicación de nuevas versiones.

Ver Figura 7 en el repositorio GitHub.

La publicación Blue-Green representada en la Figura 7 mantiene dos ambientes durante la actualización. Blue atiende a los usuarios y Green recibe la versión que será validada. Tras comprobar los flujos principales y sus conexiones, el tráfico puede dirigirse a Green; si las verificaciones fallan, las solicitudes regresan a Blue. Este retorno controlado contribuye al objetivo de recuperación del RNF-05, siempre que las migraciones de datos sean compatibles con ambas versiones.

5.4 Evaluación arquitectónica mediante Mini-ATAM

Escenario de Calidad

Durante una jornada promocional se habilita la reserva de salas de reuniones con descuento. En pocos minutos se presenta un incremento considerable de usuarios consultando disponibilidad y creando reservas de manera simultánea.

Estímulo

Incremento repentino en la cantidad de solicitudes de reserva.

Fuente del estímulo

Usuarios del coworking.

Entorno

Sistema operando en producción durante horario normal de servicio.

Respuesta esperada

El 95 % de las consultas de disponibilidad debe responder en menos de 2 segundos y las reservas confirmadas no deben presentar superposición para un mismo espacio e intervalo. Al mismo tiempo, el control de acceso debe continuar respondiendo en menos de 1 segundo, aunque Reservas esté atendiendo el aumento de solicitudes.

Punto de Sensibilidad

Servicio de Reservas y capacidad de la base de datos.

La respuesta depende de la capacidad del servicio de Reservas y de la velocidad con la que Azure SQL Database consulta y actualiza la disponibilidad. Aumentar instancias del servicio puede absorber más solicitudes, pero también puede elevar la cantidad de operaciones concurrentes sobre la base de datos. Si las consultas o los bloqueos no se controlan, el cuello de botella se desplaza hacia la capa de datos.

Trade-off Identificado

Escalabilidad versus Complejidad Operativa

La separación en microservicios permite asignar más capacidad al servicio de Reservas sin aumentar los recursos de Facturación o Notificaciones. A cambio, la operación necesita métricas por servicio, trazas con un identificador común y reglas para coordinar cambios de estado entre Membresías, Reservas y Pagos. El beneficio de escalar solo un componente debe compararse con este esfuerzo adicional de observación y soporte.

Riesgo Arquitectónico

Una configuración insuficiente del servicio de Reservas o de Azure SQL Database puede elevar los tiempos de respuesta y retrasar las confirmaciones. Bajo concurrencia alta también existe el riesgo de que dos solicitudes intenten asignar el mismo espacio antes de que la primera transacción termine, por lo que el escalamiento debe conservar los controles de consistencia.

Estrategia de Mitigación

- Ejecutar pruebas de carga con 500 usuarios concurrentes y medir por separado el tiempo del servicio y el tiempo de las consultas a la base de datos.
- Configurar reglas de escalamiento para Reservas a partir de métricas de uso y latencia verificadas durante las pruebas.
- Aplicar una validación transaccional antes de confirmar cada reserva para impedir superposiciones sobre el mismo espacio.
- Registrar métricas, errores y trazas con un identificador de correlación que permita seguir cada solicitud entre servicios.

Resultado de la Evaluación

La evaluación indica que la arquitectura puede atender el escenario porque Reservas escala de forma independiente y una falla de Notificaciones no interrumpe la operación principal. Aun así, el diseño no garantiza por sí solo el tiempo establecido en el RNF-02: Azure SQL Database continúa siendo un punto sensible y la concurrencia puede comprometer el rendimiento y la consistencia. Antes de una implementación real se deben ejecutar pruebas de carga, ajustar las consultas de mayor consumo y verificar la

validación transaccional propuesta para mitigar la duplicidad de reservas de la sección 2.3. La viabilidad del escenario depende de que estas pruebas confirmen los tiempos de respuesta y la ausencia de superposiciones.

Conclusión

La propuesta parte de una necesidad concreta: administrar reservas, membresías, cobros y accesos sin que una falla externa detenga toda la operación del coworking. El análisis de requisitos permitió reconocer que Reservas y Control de Acceso son los componentes más sensibles, mientras que Pagos y Notificaciones dependen de proveedores cuyos tiempos de respuesta no controla la aplicación.

Frente a este escenario, la arquitectura de microservicios ofrece una separación útil: permite escalar Reservas de forma independiente, mantener disponible la validación de ingreso y aislar los cambios asociados con la pasarela de pagos. La decisión no se tomó únicamente por la posibilidad de crecimiento; también se consideraron sus costos, entre ellos la necesidad de coordinar estados distribuidos, centralizar la observabilidad y automatizar los despliegues.

Las vistas 4+1 permitieron comprobar la coherencia entre los escenarios de uso, los límites de cada servicio, la secuencia de reserva, la organización del desarrollo y el despliegue en Azure. La vista de datos y la estrategia Blue-Green complementan esta descripción al definir responsabilidades sobre la información y una forma controlada de publicar nuevas versiones.

El Mini-ATAM mostró que la propuesta es viable como diseño, pero también dejó un límite claro: separar servicios no garantiza por sí mismo los tiempos de respuesta ni evita reservas superpuestas. Antes de una implementación real será necesario ejecutar pruebas con la concurrencia prevista, revisar las consultas sobre Azure SQL Database y validar los controles transaccionales y de recuperación. Con esas comprobaciones, la arquitectura puede servir como base para evolucionar el sistema sin perder trazabilidad sobre las decisiones adoptadas.

Referencias

- Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3rd ed.). Addison-Wesley.
- Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12(6), 42–50.
- Rozanski, N., & Woods, E. (2012). *Software systems architecture: Working with stakeholders using viewpoints and perspectives* (2nd ed.). Addison-Wesley.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

ISO/IEC/IEEE. (2011). *ISO/IEC/IEEE 42010:2011, Systems and software engineering—Architecture description*.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture*. O'Reilly Media.